ISOCLINE

ISOCLINE RESEARCH

Local First, Private by Design:

The Memory Architecture of Isocline

Starla Cortes

August 2026

Abstract

We present the memory architecture of Isocline, a mobile application for hormone replacement therapy monitoring—the full persistence stack, state-management framework, and privacy model that together determine where user data lives, how it is protected, and what guarantees attach to it.

The architecture rests on a six-layer persistence system: SwiftData with CloudKit for structured longitudinal data; encrypted UserDefaults for health-identifiable configuration; the Apple Keychain for the AES-256-GCM encryption key; in-memory caches with documented invalidation strategies for derived state; file-system resources for baked geometry data; and UNUserNotificationCenter for system-level reminder state. Six singleton managers, all `@MainActor @Observable` and injected via SwiftUI Environment, each own their domain exclusively and persist via a `didSet`-driven pattern that eliminates save-to-crash windows.

We describe the cold-launch sequence from container creation through manager initialization and derived-state computation; document every persistence key with its encryption status and rationale; walk through the encryption model from key generation in the Keychain through AES-GCM envelopes on UserDefaults values to the automatic migration of legacy unencrypted data; and validate the architecture against eight persistence scenarios that exercise cold launch, migration, sync conflicts, storage failure, developer-safety mechanisms, and encryption-key loss.

The paper is written to be checkable: every persistence decision is cited to its source in the codebase, every encryption boundary is named, and every limitation—from the one-way data export to the key-loss-on-restore problem—is stated plainly.

Keywords: mobile health, data persistence, encryption, privacy, SwiftData, state management, iOS security, CloudKit

Contents

1 Introduction **3**
- 1.1 The Persistence Problem . . . 3
- 1.2 The Architecture Response . . . 3
- 1.3 Contributions . . . 4

2 Design Principles **4**
- 2.1 Defense in Depth . . . 4
- 2.2 Single Source of Truth per Manager . . . 4
- 2.3 Persist at Mutation . . . 5
- 2.4 Migrate, Never Lose . . . 6
- 2.5 Developer Safety Is Production Safety . . . 6

3 The Persistence Architecture **7**
3.1 Cold Launch Sequence . . . 7
3.2 SwiftData Schema . . . 7
3.3 UserDefaults with Encryption . . . 8
3.4 The Keychain and AES-GCM . . . 9
3.5 In-Memory Caches and Derived State . . . 9
3.6 File-System Resources . . . 10
3.7 State Rehydration . . . 10
3.8 Developer State Isolation . . . 10
3.9 Data Export . . . 11

4 Encoded Architecture Decisions **11**
4.1 Persistence Patterns . . . 11
4.2 Encryption Boundaries . . . 11
4.3 Validation Gates . . . 11
4.4 Data Flow . . . 11
4.5 Keychain Accessibility . . . 13

5 Validation: Persistence Scenarios **13**

6 Discussion: Design Trade-offs and Limitations **15**
6.1 The Split-Persistence Trade-off . . . 15
6.2 DidSet Persistence vs. Periodic Save . . . 16
6.3 Symmetric Key in Keychain vs. Data Protection API . . . 17
6.4 SwiftData + CloudKit vs. Local-Only Store . . . 17
6.5 What Remains Unaddressed . . . 17
6.6 Architecture Confidence Hierarchy . . . 18

7 Conclusion **18**

1 Introduction

A mobile health application that records medication history, blood test results, injection-site locations, journal entries, and user profile data must answer a set of questions that are simple to state and difficult to satisfy simultaneously: Where does each piece of data live? How is it protected? What happens when the app restarts, when the device is lost, when the user upgrades their phone, when iCloud sync conflicts, when the encryption key is evicted from the Keychain?

Isocline answers these questions with a persistence architecture that is the subject of this paper. The application's user experience and pharmacokinetic model are described in companion papers [1, 2]; this paper concerns the *memory* that holds them together—the six-layer persistence stack, the six-manager state framework, the encryption model, and the privacy architecture that together determine what the application remembers, what it protects, and what it deliberately forgets.

1.1 The Persistence Problem

Health-tracking applications face constraints that general-purpose apps do not. A dose record entered in January must survive an app update in March, an OS upgrade in June, and a device migration in December. A medication type and ester combination directly identifies a user's medical regimen and must be encrypted at rest. A blood test calibration factor derived from the user's own lab results must be applied to every subsequent prediction without the user needing to understand it. None of this data can leave the user's control—the app has no server, no analytics, no third-party SDKs—which means the persistence architecture must be self-contained within the device and the user's own iCloud account.

These constraints produce a set of architectural requirements that shape every persistence decision in the application:

1. **Durability.** Data must survive app restarts, OS upgrades, and device migrations without user intervention.
2. **Encryption at rest.** Health-identifiable data must be encrypted such that even a forensic image of the device file system does not expose the user's medication regimen.
3. **No server dependency.** All data lives on-device or in the user's iCloud account. No Isocline server exists, so no server can be compromised.
4. **Cross-device sync.** Users with multiple devices (iPhone, iPad) should see consistent data without manual export/import.
5. **Developer safety.** Developer tooling (sample profiles, debug data seeding) must never destroy real user data.
6. **Honest degradation.** When storage fails—corrupt SwiftData store, evicted Keychain key—the app must degrade gracefully rather than crashing or silently corrupting data.

1.2 The Architecture Response

Isocline's memory architecture addresses these requirements through three structural decisions that operate at every layer:

Tiered storage by data sensitivity. Data is placed at the least permissive tier that can serve its access pattern. Structured longitudinal data (dose records, journal entries, blood tests) lives in SwiftData with CloudKit sync. Health-identifiable configuration (medication type, ester, dose schedule) lives in UserDefaults encrypted with AES-256-GCM. The encryption key lives in the Keychain with `kSecAttrAccessibleWhenUnlockedThisDeviceOnly`. Non-sensitive preferences (reminder times, UI toggles) live in plain UserDefaults. No single store holds all data; no store holds data it should not.

Singleton managers as single sources of truth. Six managers, each an `@MainActor @Observable` singleton, each owning its domain exclusively. Mutation triggers persistence

through a `didSet` pattern, not a periodic save loop. Views observe manager state through the SwiftUI Environment; they never touch storage directly. The manager is the API; the store is an implementation detail.

Defense in depth, not defense in breadth. The encryption key is generated on-device, stored in the Keychain, and never transmitted. The encrypted values in UserDefaults are opaque blobs without the key. The Keychain accessibility attribute prevents access when the device is locked and prevents the key from being included in unencrypted backups. Each layer protects against a different threat; no single-layer failure compromises all data.

1.3 Contributions

1. A complete walkthrough of the six-layer persistence stack, with code-level citations for every store and retrieval path, from the `ModelContainer` creation in `xTApp.swift` to the `DataExporter`'s multi-section CSV.
2. A documented encryption model: AES-256-GCM key generation in the Keychain, selective encryption of UserDefaults values, and automatic migration of legacy unencrypted data—with the explicit boundary between what is encrypted and what is not, and why.
3. A manager-architecture pattern (six `@MainActor @Observable` singletons, Environment-injected) with documented rehydration, invalidation, and migration paths, including the `ProfileSnapshot` symbolic-swap mechanism for developer safety.
4. An in-memory caching strategy document: which derived values are cached (`heatMapCache`, `screeningLog`), which are deliberately not memoized (`calibrationFactor`), and how the `suppressRefresh`/`performBatch` guard prevents quadratic recomputation during bulk operations.
5. An honest accounting of memory-architecture limitations: the symmetric-key-loss problem, the one-way data export, the split-persistence trade-off, the absence of integrity verification, and the structural gaps that would require architectural changes to close.

2 Design Principles

This section describes the five principles that govern every persistence decision in Isocline. They are the memory analog of the UX design principles described in the companion paper [1]: every store, every key, every encryption boundary is answerable to at least one of them.

2.1 Defense in Depth

Every piece of data in Isocline is stored at the least-permissive tier that can serve its access pattern. The six tiers are:

The architecture's central claim is that no data should live above its sensitivity floor. Medication selection—the tuple of (hormone type, ester, administration method)—directly identifies the user's medical regimen. It resides in Tier 2 (encrypted UserDefaults). Reminder times—the hour and minute at which a notification fires—carry no clinical information. They reside in Tier 4 (plain UserDefaults). The tier assignment is explicit and auditable: every UserDefaults key is listed in Table 4 with its encryption status and rationale.

2.2 Single Source of Truth per Manager

Each of the six managers owns its domain exclusively. No two managers write to the same persistent key. The `mm.` prefix on `MedicationManager`'s UserDefaults keys enforces this at the naming level; the SwiftData model context is injected from `xTApp` rather than fetched globally, ensuring the container's lifecycle is explicit and its ownership unambiguous.

Tier	Technology	Accessibility	Data Types
1. Encryption Key	Keychain	When unlocked, this device only	AES-256-GCM symmetric key
2. Encrypted Config	UserDefaults + AES-GCM	When unlocked	Medication type, ester, dose dates, profile name, profile photo
3. Structured Data	SwiftData + CloudKit	Any time (CloudKit encrypted)	Dose records, journal entries, blood tests, screenings, preferences
4. Plain Config	UserDefaults	Any time	Reminder times, UI toggles, developer flags, transient prefs
5. In-Memory Caches	`@Observable` properties	Session only	Derived state: PK curve, heat maps, adherence rate, screening log
6. System State	UNNotificationCenter	System-managed	Pending reminder requests

Table 1: The six-tier persistence architecture. Tiers are ordered by permissiveness: Tier 1 (Keychain) is the most protected; Tier 6 (UNNotificationCenter) is the least controlled by the application.

All six managers are injected via SwiftUI Environment in `xTApp.swift` (lines 139–145). The injection is explicit: each manager is a `static let shared` singleton, and the environment key's `defaultValue` returns that shared instance. Views access managers through `@Environment(medicationManager)` rather than through global singleton access, which keeps the dependency graph visible and makes it possible to inject test doubles in previews and unit tests.

2.3 Persist at Mutation

Mutation triggers persistence through a `didSet` pattern, not a periodic save loop or a `scenePhase` hook. When `selectedMedication` changes, `persistSelection()` fires immediately (`MedicationManager+Persi` 80). When `doseHistory` is modified, `saveContext()` commits to SwiftData (`MedicationManager+Persistence.s` 144). There is no window between mutation and save where data can be lost on crash.

The pattern has a cost: every individual property mutation writes to UserDefaults, which is a synchronous plist serialization. Rapid mutations would amplify this cost. The `performBatch(_:)` method (`MedicationManager.swift`:178–186) addresses this by wrapping bulk operations (e.g., seeding 110 sample doses) in a closure that suppresses `refreshState()` until the batch completes, so the PK model is recomputed once rather than 110 times:

```
func performBatch(_ work: () -> Void) {
    suppressRefresh = true
    work()
    suppressRefresh = false
    refreshState()
}
```

The pattern is the persistence analog of the UX principle "state facts, don't advise" (Section 2.1 of [1]): the manager states the mutation; the `didSet` makes it durable. The view never asks for a save; the manager never waits for one.

Manager	Files	Persistent Stores	Key Responsibilities
`MedicationManager`	7 files (main + 6 extensions)	SwiftData + encrypted UserDefaults	PK model, dose history, blood tests, calibration, site tracking
`JournalManager`	1 file	SwiftData	Mood entries, symptoms
`ProfileManager`	1 file	encrypted UserDefaults	Name, photo, date of birth
`AuthenticationManager`	1 file	plain UserDefaults	Biometric auth, auto-lock
`NotificationManager`	1 file	plain UserDefaults + UNNotificationCenter	Reminder scheduling
`SecurityManager`	1 file	Keychain	Encryption/decryption

Table 2: The six managers and their persistence responsibilities. All are `@MainActor @Observable` except `SecurityManager`, which is `Sendable`—a pure cryptographic service with no observable state.

2.4 Migrate, Never Lose

Migration from unencrypted to encrypted storage is automatic and lossless. The pattern—implemented identically in both `MedicationManager` (`Persistence.swift`:146–149) and `ProfileManager` (`ProfileManager.swift`:196–216)—is:

1. Try reading the encrypted key.
2. If absent, fall back to the unencrypted key.
3. If the unencrypted key succeeds, write the encrypted version and remove the plaintext.

The migration is exercised on every cold launch via `migrateToEncryptedStorage()`, which is idempotent: it checks for the existence of the encrypted key before writing. A user who upgrades from a build that stored medication selection in plain `selectedMedication` to a build that stores it in `encryptedMedicationSelection` will have their data transparently upgraded on first launch, with no data loss and no prompt.

The migration also operates in the opposite direction for *reading*: `loadSelection()` (`Persistence.swift`:572) first attempts the encrypted key; on miss, falls back to the legacy plaintext key. This means the migration does not need to complete before the data is usable—the app reads the legacy key if the migration has not yet run, and the migration runs on the next write.

2.5 Developer Safety Is Production Safety

Developer tooling must never destroy real user data. Two mechanisms enforce this:

Developer settings isolation. All developer flags live under a `dev.` prefix in UserDefaults (`DeveloperSettings.swift`:27–29). In DEBUG builds, these flags are readable and writable. In RELEASE builds, they are hardwired `false` and their setters are compiled out—no code path in a RELEASE build can set a developer flag. On RELEASE launch, `scrubStaleReleaseState()` (`DeveloperSettings.swift`:70–76) removes every `dev.*` key left by a prior DEBUG install, preventing a RELEASE-over-DEBUG install from inheriting stale developer state.

Symbolic swap for sample profiles. The developer sample profile (“Jan”) seeds 110 synthetic dose records, blood tests, and journal entries for testing. Enabling it must not destroy the user’s real data. `ProfileSnapshot` (DEBUG-only, `ProfileSnapshot.swift`) implements a “symbolic swap”: before Jan’s data is seeded, `snapshotIfNeeded()` captures every UserDefaults key and every SwiftData record into a JSON file in the developer support directory (`xT-dev/user-snapshot.json`). When the sample profile is disabled, `restore()` wipes the live store, reinserts the captured records, and calls `rehydrateManagers()` to reset all in-

memory state. The swap is "symbolic" because only records move—the CloudKit store itself is never duplicated or replaced.

The `DosePayload` struct carries all 17 fields of `DoseRecord`, including the injection-site JSON and the dose-correction audit trail (`isUserCorrected`, `originalDoseId`, `originalSiteJSON`). A comment at line 266 warns: "any field missing here is destroyed permanently the first time the user toggles the Sample profile off."

3 The Persistence Architecture

This section walks through each layer of the persistence stack, tracing the lifecycle of data from the moment the app launches to the moment it is written to durable storage. It is the memory analog of the UX paper's "User Journey" (Section 3 of [1]).

3.1 Cold Launch Sequence

The cold-launch sequence in `xTApp.swift` establishes the persistence stack in five steps:

1. **ModelContainer creation** (lines 54–87): A static `ModelContainer` is initialized with the schema of five `@Model` types and `cloudKitDatabase: .automatic`. If creation fails, the app falls back to an in-memory store—the on-disk store is never deleted, so a transient error (file corruption, lock contention) gets another chance on the next launch.
2. **Context wiring** (lines 91–92): `MedicationManager.shared.modelContext` and `JournalManager.shared.mod` receive the container's `mainContext`. The `modelContext` property's `didSet` triggers deferred loading: `JournalManager` calls `loadEntries()` when the context arrives.
3. **UserDefaults registration** (lines 97–106): All keys that are read but not yet written are registered with default values, preventing the "found no value for key" log noise that would otherwise accompany every cold launch before the user has configured reminders.
4. **Manager initialization**: Each manager's `init()` calls its `load*()` methods. UserDefaults-backed state loads synchronously. SwiftData-backed state loads when the context arrives via `didSet`.
5. **Derived state computation**: `MedicationManager.refreshState()` recomputes the PK curve, adherence rate, cycle position, and hormone level estimate from the loaded data.

The sequence is deterministic and synchronous where possible. There is no asynchronous loading spinner at launch; the data is either available immediately (UserDefaults) or available within one run-loop turn (SwiftData's mainContext fetch).

3.2 SwiftData Schema

The SwiftData schema was chosen over Core Data for three reasons: CloudKit sync is a single configuration flag rather than a multi-step NSPersistentCloudKitContainer setup; the `@Model` macro eliminates the need for separate `.xcdatamodeld` files; and SwiftData's `FetchDescriptor` API is native Swift rather than an Objective-C bridge.

Soft-delete pattern. `DoseRecord` carries an `isDeleted` flag rather than being removed from the store. This preserves the audit trail for dose corrections: a corrected dose retains its `originalDoseId` reference to the original record, which must still exist in the store. The `loadDoseHistory()` method (`Persistence.swift`:132–138) filters out soft-deleted records:

```
doseHistory = (allRecords ?? []).filter { !$0.isDeleted }
```

CloudKit fallback. If the on-disk `ModelContainer` fails to initialize, the app falls back to `isStoredInMemoryOnly: true` (`xTApp.swift`:78–86). The in-memory store survives the session but not the launch. Critically, the code does *not* delete the on-disk store:

```
// Leave the on-disk store untouched so user data is never actively destroyed.
// A transient error gets another chance on the next launch.
```

Model	Key Fields	CloudKit	Notes
`DoseRecord`	17 stored properties: id, doseDate, medicationType, ester, administrationMethod, doseAmount, concentrationMgPerMl, notes, isDeleted, doseStatus, skipReason, injectionTime, injectionSitesJSON, recordedAt, isUserCorrected, originalDoseId, originalSiteJSON	Synced	Soft-delete via `isDeleted` flag; injection sites stored as raw JSON
`JournalEntry`	id, date, mood, text, symptoms (JSON), hormoneLevel	Synced	Symptoms encoded as JSON array of strings
`BloodTestRecord`	id, drawDate, measuredLevel, hormoneType, hoursSinceLastDose, notes, calibrationFactor (transient)	Synced	`calibrationFactor` is recomputed on load, not persisted
`ScreeningRecord`	id, date, typeRawValue, notes	Synced	Screening type stored as raw String for forward compatibility
`UserPreferences`	medicationRow, medicationColumn, lastDoseDate, nextDoseDate, concentrationDefaults, suppressionSpeed, spironolactoneClearanceEnabled, customFrequencyDays, defaultDoseAmount, defaultInjectionTime, showDoseIntensityInsights	Synced	One record per user; fetched as `FetchDescriptor.first`

Table 3: The five SwiftData `@Model` types. All participate in CloudKit sync via `cloudKitDatabase: .automatic`.

This means a corrupt store will produce session-only data loss rather than permanent data destruction. The user is not explicitly notified—the app launches normally, and data entered during the session is lost on next launch—which is a known limitation (Section 6.5).

3.3 UserDefaults with Encryption

Key	Enc?	Manager	Type	Purpose
`encryptedMedicationSelection`	Yes	Medication	pipe-delimited String	Medication type | method | ester
`encryptedDoseDates`	Yes	Medication	JSON container	`{lastDoseDate, nextDoseDate, hrtStartDate}`
`encryptedProfileName`	Yes	Profile	UTF-8 String	User's chosen display name
`encryptedProfileImageData`	Yes	Profile	JPEG Data	Profile photo ($\leq$ 5 MB)
`profileDateOfBirth`	No	Profile	Date	Demographic, not health-identifiable
`mm.antiAndrogenInUse`	No	Medication	Bool	Whether an anti-androgen is configured
`mm.spironolactoneInUse`	No	Medication	Bool	Specific anti-androgen flag
`mm.spironolactoneClearanceEnabled`	No	Medication	Bool	Clearance adjustment toggle
`mm.suppressionSpeed`	No	Medication	String	Endogenous taper rate (fast/medium/slow)
`mm.customFrequencyDays`	No	Medication	Int?	User-set dosing interval override
`mm.missedDoseGracePeriodHours`	No	Medication	Double	Hours before a dose is marked missed

carries no clinical information. The `mm.*` transient preferences (anti-androgen flags, suppression speed, therapeutic targets) are not encrypted because they are numerical or boolean configuration values that, individually, do not identify a medical regimen.

This is a judgment call. A user's spironolactone flag combined with their custom frequency days and therapeutic targets *could* reveal clinical information in aggregate. The current design accepts this risk on the reasoning that the combination requires physical device access, at which point the attacker already has the device. A future iteration might move all `mm.*` keys to encrypted storage; the migration path already exists.

3.4 The Keychain and AES-GCM

`SecurityManager` (`SecurityManager.swift`, 97 lines) is a `Sendable` singleton that manages the application's data-at-rest encryption. It is the only manager that is not `@Observable`—it has no observable state, only a cryptographic API.

Key generation. On first access, `getEncryptionKey()` generates a 256-bit AES-GCM symmetric key via `SymmetricKey(size: .bits256)` from Apple's CryptoKit. The key is serialized and stored in the Keychain under service `com.starlacortes.xT.security`, account `xTEncryptionKey`. If the key already exists in the Keychain, it is fetched rather than regenerated.

Keychain accessibility. The key is stored with `kSecAttrAccessibleWhenUnlockedThisDeviceOnly`. This attribute means:

- The key is only accessible when the device is unlocked (first unlock after boot is sufficient).
- The key is *not* included in unencrypted iTunes/Finder backups.
- The key is *not* synced to other devices via iCloud Keychain (`ThisDeviceOnly`).
- The key may be evicted on device restore if the backup was not encrypted.

The choice of `ThisDeviceOnly` is deliberate: the encryption key should never leave the device that generated it. But it creates the key-loss problem exercised by scenario Nadia (Section 5) and documented as a structural limitation (Section 6.5).

Encryption API. `encryptData(_:)` uses `AES.GCM.seal` with a system-generated 96-bit nonce, returning the `combined` representation (nonce || ciphertext || tag). `decryptData(_:)` reconstructs a `SealedBox` from the combined representation and calls `AES.GCM.open`. The combined format is a standard: the nonce is prepended, so each encryption is independent and there is no nonce-reuse risk across encryptions.

Error handling. The `SecurityError` enum has five cases: `keychainSaveFailed`, `keychainReadFailed`, `encryptionFailed`, `decryptionFailed`, and `sealedBoxUnavailable`. The last is a safety net: the code uses `guard let combined = sealedBox.combined` rather than a force-unwrap, because "health data is not worth a force unwrap" (comment at `SecurityManager.swift`:41).

3.5 In-Memory Caches and Derived State

The managers hold in-memory mirrors of persisted data for reactive SwiftUI binding, plus derived state that is computed from that data. The caching strategy distinguishes three categories:

Direct mirrors (loaded from store, mutated via manager methods): `doseHistory`, `bloodTestHistory`, `journalEntries`. These are loaded on init or context arrival and kept in sync with the store by the `didSet` → `persist` pattern.

Computed derived state (recomputed by `refreshState()`): `currentHormoneLevel`, `recentLevels`, `adherenceRate`, `onTimeStreak`, `regimenTransitionHistory`. These are pure functions of the loaded data; they are recomputed whenever the underlying data changes.

Cached computed state with explicit invalidation:

- `heatMapCache` (`MedicationManager+Sites.swift`): A `@ObservationIgnored` dictionary mapping `"routeClass_windowDays"` keys to per-zone injection-count dictionaries. Invalidated

after each dose record via `invalidateHeatMapCache()`. Computation is offloaded to a `Task.detached(priority: .utility)` to avoid blocking the main actor.
- `screeningLog` (`MedicationManager.swift`): A compact `[ScreeningType: Date]` lookup table, rebuilt from `screeningRecords`'s `didSet` observer. Marked `@ObservationIgnored` because it is only accessed programmatically, not rendered directly by SwiftUI views.

Deliberately not memoized: `calibrationFactor`. A comment at `MedicationManager.swift`:259–262 explains: "there is no mutation hook a cache could hang invalidation off." The calibration factor depends on blood test results, PK model constants, and dose history—any of which can change independently. Rather than tracking every invalidation source, the factor is recomputed on demand.

Batch mutation guard. The `suppressRefresh` flag and `performBatch(_:)` method prevent the quadratic cost of recomputing the PK model after each of n bulk dose insertions. During sample profile seeding (110 doses), `suppressRefresh` is set to `true`, all doses are inserted, and `refreshState()` runs once at the end.

3.6 File-System Resources

Two categories of data live on the file system rather than in SwiftData or UserDefaults:

Zone maps (`ZoneMapFile.swift`). Baked per-vertex zone identification data for the 3D body model is stored as JSON resources in the app bundle (`zone_map_<bodyType>.json`). Each zone map assigns one `UInt16` zone ID to every vertex of the USDZ mesh (∼14,000 vertices), encoded as a little-endian byte blob. The file carries a `schemaVersion` field; `ZoneMap` refuses to load a file whose version it does not recognize rather than silently mislabeling taps. Vertex count is validated against the loaded mesh—a mismatch would scramble every label. The previous design resolved taps by scanning 49 centroids; 57% of the mesh fell outside the centroid cutoffs. The per-vertex approach replaced a heuristic with a lookup.

Developer snapshot (`ProfileSnapshot.swift`, DEBUG-only). Captured to `xT-dev/user-snapshot.json` in the application support directory, deliberately outside the CloudKit-backed SwiftData store so developer state can never sync. The snapshot contains all UserDefaults keys (as a binary plist for lossless round-tripping) and all SwiftData records (as Codable payloads). Restore wipes the live store, reinserts captured records, and calls `rehydrateManagers()`.

3.7 State Rehydration

`ProfileSnapshot.rehydrateManagers()` (`ProfileSnapshot.swift`:196–247) is the single code path that resets the application to a cold-launch state without restarting the process. It:

1. Resets every `ProfileManager` property to empty, then calls `loadProfile()`.
2. Resets every `MedicationManager` property to its cold-launch default (nil medication, empty histories, nil hormone level), reloads transient preferences from UserDefaults, then calls `loadSelection()`, `loadDoseDates()`, `loadDoseHistory()`, `loadBloodTestHistory()`, `loadScreeningHistory()`, `refreshState()`, and `recomputeCalibrationFactors()`.
3. Resets `JournalManager`'s symptoms and calls `loadEntries()`.
4. Resets `NotificationManager`'s reminder settings from UserDefaults and calls `scheduleAllReminders()`.

The rehydration sequence mirrors the cold-launch `init()` sequence exactly, which means it exercises the same code paths. If `rehydrateManagers()` works, cold launch works—there is no separate "hot reload" code path that could drift from the launch path.

3.8 Developer State Isolation

Developer settings live under a `dev.` prefix in UserDefaults (`DeveloperSettings.swift`:27–29) and a dedicated support directory (`xT-dev/`). Three mechanisms protect production state from developer tooling:

1. **Prefix isolation.** Every developer key carries the `dev.` prefix. `scrubStaleReleaseState()` iterates the UserDefaults dictionary and removes every key matching `dev.*`. No production key has this prefix.
2. **Compilation gating.** In RELEASE builds, the `developerModeEnabled` and `sampleProfileEnabled` properties are hardwired `let` constants set to `false`. Their `didSet` persistence code is compiled out. No code path in a RELEASE build can set a developer flag.
3. **Directory isolation.** The developer support directory (`xT-dev/`) is outside the CloudKit-scoped SwiftData store, so snapshot files can never sync to iCloud—even if a DEBUG build were to accidentally leave a snapshot file in place.

3.9 Data Export

`DataExporter` (`DataExporter.swift`, 439 lines) is a stateless formatter that produces portable CSV and JSON from live manager state. It does not read raw stores; it aggregates state through an `ExportState` struct that the caller populates from the managers.

The CSV export is multi-section: dose records, injection sites (one row per site per dose), journal entries, blood tests, avoid zones, and user preferences—each as a labeled section separated by blank lines. The JSON export is a single `ExportPayload` with nested arrays for each record type.

Locale independence. The export filename uses `AppDateFormatters.isoDay`, which is POSIX- and UTC-locked. A device on a Buddhist or Japanese calendar still writes `xT_doses_2026-08-04.cs` rather than `xT_doses_2569-08-04.csv`. The comment at line 249 explains: "Filenames must not vary by locale or calendar."

One-way export. The `DataExporter` produces CSV and JSON but has no corresponding `DataImporter`. A user who exports their data cannot reimport it. This is a known limitation (Section 6.5).

4 Encoded Architecture Decisions

This section documents the persistence patterns, encryption boundaries, and validation gates that ship in the application. It is the memory analog of the UX paper's "Encoded Design Decisions" (Section 4 of [1]) and the PK paper's "Encoded Constants" (Section 3 of [2]): a reference for what is in the codebase, where it came from, and what confidence attaches to it.

4.1 Persistence Patterns

4.2 Encryption Boundaries

The encryption boundary is a conscious split. SwiftData records (dose history, journal entries, blood tests) are not additionally encrypted by the application because: (a) CloudKit encrypts data in transit (TLS) and at rest (on Apple's servers); (b) the records are structural and benefit from CloudKit's sync, query, and merge capabilities; (c) adding an application-layer encryption envelope over SwiftData records would prevent CloudKit from performing server-side merge operations. The trade-off is that Apple holds the CloudKit encryption keys, not the user—a limitation discussed in Section 6.5.

4.3 Validation Gates

4.4 Data Flow

The reactive data pipeline flows in one direction, with no cycles:

Pattern	How It Works	Where
`didSet` → persist	Property observer calls a `persist*()` method on mutation	`MedicationManager.swift`, `AuthenticationManager.swift`
`load` → hydrate	`init()` calls `load*()` methods; SwiftData load deferred to `modelContext didSet`	All managers
`encrypt` → envelope	`SecurityManager.encryptData()` wraps value in AES-GCM combined blob	`SecurityManager.swift`
Auto-migration	On load, try encrypted key; on miss, try legacy plaintext; on hit, write encrypted, delete plaintext	`MedicationManager+Persistence.swift:146`, `ProfileManager.swift:196`
SwiftData `FetchDescriptor`	`FetchDescriptor<Model>(sortBy: [...])` → `context.fetch()`	`Persistence.swift:132–138`
Soft delete	Set `isDeleted = true`; filter on load	`DoseRecord.swift:31`, `Persistence.swift:137`
Deferred load	`modelContext didSet` triggers `loadEntries()` or equivalent	`JournalManager.swift:21`, `MedicationManager.swift:117–122`
Symbolic swap	Snapshot all records to JSON; wipe live store; reinsert from JSON; rehydrate managers	`ProfileSnapshot.swift:141–191`
Batch guard	`suppressRefresh = true`; perform work; `suppressRefresh = false`; `refreshState()` once	`MedicationManager.swift:178–186`

Table 5: The nine persistence patterns used across the codebase. Each pattern has a single canonical implementation; no pattern is reinvented in a second location.

```
User Action
    |
    v
@Observable property mutation (Manager)
    |
    +---> didSet ---> persist*() ---> SwiftData / UserDefaults
    |
    +---> refreshState() (if not suppressed)
              |
              v
         Derived State (currentHormoneLevel, recentLevels,
                        adherenceRate, regimenHistory)
              |
              v
         SwiftUI Views (re-render via @Observable)
```

Views observe manager state through `@Environment` and SwiftUI's `@Observable` tracking. They never call `save()` or `load()`—the manager is the API, and persistence is an implementation detail of property mutation. A view that sets `medicationManager.selectedMedication = "Estrogen"` does not know or care that this triggers AES-GCM encryption and a UserDefaults write; it mutates a property and the manager handles the rest.

Data	Encrypted?	Rationale
Medication type, ester, method	Yes	Directly identifies HRT regimen
Dose dates (last, next, HRT start)	Yes	Reveals treatment timeline and start date
Profile name	Yes	May identify user personally
Profile photo	Yes	Face image, highest sensitivity
Date of birth	No	Demographic, not uniquely identifying
Reminder times	No	Clock times carry no clinical information
Anti-androgen flag	No	Binary toggle, low individual sensitivity
Suppression speed	No	“fast”/“medium”/“slow” enum value
Custom frequency days	No	Integer; individual sensitivity low
Therapeutic targets	No	User-set numerical ranges
Dose records	No (CloudKit)	Structural data; CloudKit encrypts in transit and at rest
Journal entries	No (CloudKit)	User-authored content
Blood test results	No (CloudKit)	Lab values; CloudKit encrypts in transit and at rest

Table 6: Encryption status for every category of persisted data. “No (CloudKit)” means the data relies on CloudKit’s built-in encryption rather than application-layer AES-GCM.

4.5 Keychain Accessibility

5 Validation: Persistence Scenarios

Section-level descriptions of persistence layers describe the architecture as designed. Scenarios describe the architecture as *exercised*—by specific failure modes, edge cases, and transitions. This section adopts the scenario-validation approach of the companion UX paper (Section 5 of [1]), exercising memory-architecture edge cases rather than UX edge cases.

Scenario	Event	What it tests, and the result
Maya, cold launch	Fresh install, no prior data. Completes onboarding; force-quits; relaunches.	**Cold-launch persistence.** `MedicationManager.init()` loads from empty UserDefaults. Onboarding writes medication selection, dose dates, and user preferences. On restart, all values are decrypted and loaded. SwiftData container initializes with CloudKit schema. The PK curve shows the single post-onboarding dose. Pass.
Dev, legacy migration	Upgrading from a build that stored medication selection unencrypted in the `selectedMedication` key.	**Unencrypted-to-encrypted migration.** `migrateToEncryptedStorage()` checks for `encryptedMedicationSelection`; on miss, `loadSelection()` falls back to `selectedMedication`; on hit, writes encrypted version and removes plaintext. Both `MedicationManager` and `ProfileManager` follow identical paths. Pass.

Scenario	Event	What it tests, and the result
Priya, iCloud sync conflict	Enters a dose on iPhone; later opens iPad. Both devices have pending CloudKit changes for the same record.	**CloudKit merge.** SwiftData's `.automatic` CloudKit database handles merge. `DoseRecord`'s `@Attribute(.unique) var id` prevents duplication. The `isDeleted` soft-delete flag survives merge conflicts—a record deleted on one device stays deleted after sync. Pass (delegated to SwiftData).
Sam, ModelContainer failure	File corruption or schema version mismatch prevents SwiftData container creation.	**Graceful degradation.** `xTApp.swift`:78–86 creates an in-memory store. The app launches normally. Dose records entered during the session are lost on next launch, but UserDefaults data (medication selection, profile) survives. On next launch with cleared corruption, SwiftData reinitializes from CloudKit or a fresh store. Pass (graceful degradation).
Rowan, sample profile swap	Developer toggles Sample profile on (Jan's data is seeded), uses it, toggles off.	**Symbolic swap integrity.** `ProfileSnapshot.snapshotIfNeeded()` captures all 68+ UserDefaults keys and all SwiftData records before Jan's data is seeded. On toggle-off, `restore()` wipes all live records, reinserts captured ones, and calls `rehydrateManagers()`. The user's original data is byte-identical to pre-toggle state. The `DosePayload` carries all 17 fields. Pass.
Kai, RELEASE over DEBUG	User installs App Store build over a TestFlight DEBUG build that had developer flags set (`dev.developerModeEnabled = true`).	**Developer state scrub.** `xTApp.init()` calls `scrubStaleReleaseState()` in RELEASE. All `dev.*` keys are removed. The `xT-dev/` directory is deleted. `developerModeEnabled` is hardwired `false`. No stale state survives. Pass.
Jules, data export	User exports all data via Settings → Data Export after 6 months of use.	**Export completeness.** `DataExporter` reads live manager state. CSV contains 6 labeled sections: dose records (with injection sites as separate rows), journal entries, blood tests, avoid zones, and preferences. JSON includes all fields from all model types including the dose-correction audit trail. Filename uses POSIX-locked date formatter. Export is one-way—there is no import path. Pass (export only).

Scenario	Event	What it tests, and the result
Nadia, encryption key loss	Device is restored from an unencrypted backup. The Keychain item is evicted. The app launches.	**Key eviction behavior.** `SecurityManager.getEncryptionKey()` fails to read from Keychain, generates a new key. The new key cannot decrypt existing `encryptedMedicationSelection`, `encryptedDoseDates`, `encryptedProfileName`, or `encryptedProfileImageData`. `loadSelection()` falls back to legacy unencrypted keys (which don't exist post-migration), loads empty defaults. The app launches with no medication configured—the user must re-enter their medication details and profile. Dose records and journal entries (SwiftData/CloudKit) survive. **Fail with data loss.**

Scenario Nadia is the only failure in the set. It exercises the symmetric-key model's structural limitation: a single key protects all encrypted UserDefaults values, and losing that key means losing all of them. The scenario is included because the UX paper's principle of "honest presentation" (Section 2.1 of [1]) applies equally to architecture documentation: limitations must be named, not hidden.

6 Discussion: Design Trade-offs and Limitations

This section describes trade-offs that were consciously accepted and limitations that remain unaddressed. It is the memory analog of the UX paper's Discussion (Section 6 of [1]) and the PK paper's Discussion (Section 5 of [2]).

6.1 The Split-Persistence Trade-off

Medication selection and dose dates live in encrypted UserDefaults (Tier 2), while dose records and blood tests live in SwiftData (Tier 3). This split means that a complete picture of the user's medication regimen requires reading from two stores with different encryption properties, different query capabilities, and different sync behavior.

The split is justified by three factors:

1. **Access frequency.** Medication selection is read once per launch to determine which PK constants to use. It changes only when the user changes their medication in Settings—a rare event. UserDefaults is optimized for this pattern: read-once, write-rarely, key-value.
2. **Sensitivity.** The tuple (hormone type, ester, method) directly identifies the user's medical regimen. Individual dose records are less revealing: a single record of "4 mg EV, 2026-03-15" reveals a dose amount but not the regimen structure. The sensitivity difference justifies the encryption difference.
3. **Sync requirements.** Dose records benefit from cross-device sync—a user should see their full dose history on both iPhone and iPad. Medication selection is unlikely to change between devices and does not benefit from sync. Encrypting data that needs CloudKit merge would be counterproductive: CloudKit cannot merge encrypted blobs.

Gate	Value	File	Line
Profile image max size	5 MB	`ProfileManager.swift`	30
Profile name max length	50 chars	`ProfileManager.swift`	27
JPEG compression quality	0.8	`ProfileManager.swift`	24
Calibration factor α clamp	$[0.2, 5.0]$	`MedicationManager+Dosing.swift`	
Schema version check	1	`ZoneMapFile.swift`	28
Vertex count validation	matches loaded mesh	`ZoneMapFile.swift`	66–68
Dose volume warning	> 3.0 mL	`MedicationManager+Dosing.swift`	
Injection dose sanity	≤ 1000 mg	`MedicationManager+Dosing.swift`	

Table 7: Input validation gates that prevent silent data corruption. Values outside these ranges are rejected with an explicit error or warning rather than silently persisted.

Attribute	Value	Effect
Class	`kSecClassGenericPassword`	Standard Keychain item type for arbitrary data
Service	`com.starlacortes.xT.security`	Namespaced to the app's bundle identifier
Account	`xTEncryptionKey`	Distinguishes the encryption key from other Keychain items
Accessibility	`kSecAttrAccessibleWhenUnlockedThisDeviceOnly`	Not accessible when device is locked; not included in unencrypted backups; not synced via iCloud Keychain

Table 8: Keychain attributes for the AES-256-GCM encryption key. The `ThisDeviceOnly` suffix is load-bearing: it prevents the key from appearing on other devices signed into the same iCloud account.

The cost is architectural complexity: a developer reading the codebase must understand that "medication data" is split across two stores, and a change to the medication model may require changes to both.

6.2 DidSet Persistence vs. Periodic Save

The `didSet` $\rightarrow$ `persist()` pattern writes to storage on every mutation. This trades write frequency for safety: no data is ever lost between the mutation and a periodic save. The cost is that rapid mutations would trigger excessive writes, which is why `performBatch(_:)` suppresses `refreshState()` during bulk operations.

An alternative design would use a periodic save (e.g., on `scenePhase .background`) with an in-memory change queue. This would reduce write frequency at the cost of a data-loss window between the last mutation and the next background event. For a health-tracking application where a dose record is the most important user action, that window is unacceptable: a user who records a dose and immediately force-quits must find the dose present on relaunch.

6.3 Symmetric Key in Keychain vs. Data Protection API

The current design generates a single 256-bit AES-GCM key and stores it in the Keychain. An alternative would use Apple's Data Protection API (`NSFileProtectionComplete`), which encrypts files at the OS level without application-managed keys. The symmetric-key approach was chosen because: (a) it works independently of file-level protection, allowing the app to encrypt individual UserDefaults values rather than entire files; (b) it allows selective encryption—some UserDefaults keys are encrypted, others are not; (c) the Keychain accessibility attribute `WhenUnlockedThisDeviceOnly` provides granular control over when the key is available.

The cost is key management complexity and the key-loss problem (scenario Nadia, Section 5).

6.4 SwiftData + CloudKit vs. Local-Only Store

SwiftData with CloudKit was chosen over a local-only store (Core Data, raw SQLite) because it provides cross-device sync without a server. The trade-offs: (a) CloudKit sync introduces latency and potential merge conflicts; (b) the schema is constrained by CloudKit's type system (enums stored as raw String values throughout the codebase, e.g., `ScreeningRecord.typeRawValue`); (c) the app is tied to the user's iCloud account—a user without iCloud gets no sync.

The benefit is that no server infrastructure exists, no user accounts are created, and no data leaves the user's iCloud account. For a health application, this is the right trade: "no server" means no server to breach.

6.5 What Remains Unaddressed

The following limitations are structural—they cannot be addressed by refining the existing architecture and would require new features or infrastructure.

No end-to-end encryption of SwiftData. CloudKit encrypts data in transit (TLS) and at rest on Apple's servers, but Apple holds the encryption keys. The application does not add an encryption layer over SwiftData records. A user who wants full end-to-end encryption of all health data—so that not even Apple can access it—would need a different architecture (application-layer encryption over each record, with key material that never touches Apple's servers).

Key recovery on device restore. If a user restores their device from a backup that does not include Keychain items (iTunes backup without encrypted backup enabled), the encryption key is lost. Encrypted UserDefaults values become permanently unreadable. The app does not surface this risk or offer a recovery mechanism. Mitigations exist: encrypted iTunes backups preserve Keychain items; iCloud backups always preserve them. But the app does not guide the user toward these mitigations.

No data integrity verification. The app does not compute checksums or MACs over persisted data. A corrupted UserDefaults plist or SwiftData store would produce silent errors (failed decryption, zero-length arrays) rather than explicit corruption warnings. Adding integrity verification would require a MAC over each persisted value, which would need its own key management—the current single-key model would need to become a two-key model (one for confidentiality, one for integrity).

No backup/restore beyond iCloud. The `DataExporter` produces CSV and JSON but has no corresponding `DataImporter`. A user who exports their data cannot reimport it. The export is a one-way safety hatch for the user's own records, not a migration tool.

No selective sync. All SwiftData models participate in CloudKit sync. There is no mechanism for a user to say "sync dose records but keep journal entries local-only." The all-or-nothing sync model is simpler but less flexible.

Silent fallback on decryption failure. When `loadSelection()` fails to decrypt the encrypted medication selection, it falls back to the legacy unencrypted key; if that is also absent, it loads empty defaults. The user is not notified that their encrypted data could not be read. This is a deliberate choice: surfacing a "decryption failed" error on launch would be alarming to a user who does not know what encryption is, and the most likely cause (key eviction on restore) is not something the user can fix from within the app. But it means data loss can occur silently.

Developer safety is DEBUG-only. The `ProfileSnapshot` mechanism that protects user data during sample profile testing is compiled out of RELEASE builds. If a RELEASE build ever needed a similar safety net (e.g., for a data repair tool or a migration assistant), the infrastructure would not exist.

6.6 Architecture Confidence Hierarchy

Analogous to the UX paper's design decision confidence hierarchy (Section 6.7 of [1]) and the PK paper's source quality hierarchy (Section 5.3 of [2]), persistence decisions in Isocline rest on a hierarchy of confidence:

1. **Platform-standard** (highest confidence). Decisions that use Apple frameworks as designed: SwiftData `@Model` persistence, CloudKit `.automatic` sync, Keychain Services API, CryptoKit AES-256-GCM, `UNUserNotificationCenter`, `LocalAuthentication`. These are tested across billions of devices.
2. **Security-grounded.** Decisions that follow from cryptographic best practices: on-device key generation, AES-GCM with system-generated nonces, `kSecAttrAccessibleWhenUnlockedThisDeviceO` These are the standard recommendations for data-at-rest encryption on iOS.
3. **Reasoned but untested.** Decisions based on architectural reasoning without formal performance or security testing: the `didSet` persistence pattern's behavior under rapid mutations, the encrypted UserDefaults key count's impact on cold-launch time, the `ProfileSnapshot`'s memory footprint with large dose histories (1,000+ records), the CloudKit merge behavior for `InjectionSite` JSON blobs. These are internally consistent and follow from the design principles, but their edge-case behavior has not been profiled.
4. **Placeholder** (lowest confidence). Decisions known to be suboptimal but retained because a better alternative is not yet available: the lack of key recovery on device restore, the absence of integrity verification, the one-way data export, the silent fallback on decryption failure. These are documented gaps; closing them would require architectural changes rather than incremental improvements.

7 Conclusion

We have presented the memory architecture of Isocline, a mobile application for hormone replacement therapy monitoring. The architecture's key structural features are:

- A **six-layer persistence stack** that assigns every piece of data to the least-permissive tier that can serve its access pattern: Keychain for the encryption key, encrypted UserDefaults for health-identifiable configuration, SwiftData with CloudKit for structured longitudinal data, plain UserDefaults for non-sensitive preferences, in-memory caches for derived state, and UNNotificationCenter for system-level reminders.
- An **encryption model** in which a single AES-256-GCM symmetric key—generated on-device, stored in the Keychain with `WhenUnlockedThisDeviceOnly`, and never transmitted—protects the four health-identifiable UserDefaults values, with automatic migration of legacy unencrypted data.
- A **manager architecture** of six `@MainActor @Observable` singletons, each owning its domain exclusively and persisting via a `didSet`-driven pattern that eliminates the save-to-

crash window, with a `performBatch` guard for bulk operations and documented invalidation strategies for cached derived state.
- A **developer safety system** in which `dev.*`-prefixed UserDefaults keys and a dedicated support directory isolate developer state from production data, `ProfileSnapshot` implements a symbolic swap to protect user data during sample profile testing, and `scrubStaleReleaseState()` guarantees that a RELEASE build never inherits developer flags.
- A **rehydration mechanism** (`ProfileSnapshot.rehydrateManagers()`) that resets the application to cold-launch state without restarting the process, exercising the same load paths as a genuine cold launch.

The eight persistence scenarios (Section 5) exercise the architecture across cold launch, migration, sync conflicts, storage failure, developer safety, data export, and key loss. Seven pass; one (key loss on device restore) fails with data loss, which is named as a structural limitation rather than hidden.

Design trade-offs are stated plainly: the split between encrypted UserDefaults and SwiftData reflects a sensitivity/access-frequency judgment; the `didSet` persistence pattern prioritizes safety over write efficiency; the symmetric-key model provides selective encryption at the cost of key management complexity; the CloudKit dependency provides server-free sync at the cost of reduced control over encryption key custody.

What remains unaddressed—end-to-end encryption of SwiftData records, key recovery on device restore, data integrity verification, backup/restore beyond iCloud, selective sync, and silent fallback on decryption failure—is listed without apology.

The most important property of a health-data persistence architecture, in our view, is the same as the most important property of the pharmacokinetic model that drives it and the user interface that delivers it: **honesty about what it can and cannot protect.** Isocline's memory architecture is designed to be honest. It encrypts what must be encrypted and documents what it does not. It names its key-loss scenario rather than pretending it does not exist. It states which trade-offs are conscious and which are inherited. This honesty—the explicit marking of what is protected, what is not, and why—is not the most visible property of the persistence layer, but it is the one that matters most for an application whose users trust it with their medical data.

This paper describes a mobile application's persistence architecture. It is not a security audit, and the architecture decisions described here should not be interpreted as security guarantees. The application's encryption model protects data at rest on the device but does not provide end-to-end encryption of CloudKit-synced records. The symmetric-key model has no recovery mechanism if the Keychain item is evicted.

References

[1] Cortes S. (2026). “Progressive Clarity with Honest Constraint: A Design Framework for Hormone Replacement Therapy Tracking in the Isocline Mobile Application.” August 2026.

[2] Cortes S. (2026). “Superposed Bateman Biexponentials with Clearance-Constrained Scaling and Per-User Calibration: A Pharmacokinetic Framework for Hormone Replacement Therapy Monitoring in the Isocline Mobile Application.” August 2026.

[3] Apple Inc. (2024). “CryptoKit.” Apple Developer Documentation. https://developer.apple.com/documentation/cryptokit

[4] Apple Inc. (2024). “Keychain Services.” Apple Developer Documentation. https://developer.apple.com/documentation/security/keychain_services

[5] Apple Inc. (2024). “SwiftData.” Apple Developer Documentation. https://developer.apple.com/documentation/swiftdata

[6] Apple Inc. (2024). “CloudKit.” Apple Developer Documentation. https://developer.apple.com/documentation/cloudkit

[7] Apple Inc. (2024). “Apple Platform Security.” Apple Platform Security Guide. https://help.apple.com/pdf/security/en_US/apple-platform-security-guide.pdf